

FIAP GRADUAÇÃO

DOMAIN DRIVEN DESIGN

Prof. Me. Thiago T. I. Yamamoto

#12 – TRATAMENTO DE EXCEÇÕES



TRAJETÓRIA



Collections Framework



Tratamento de Exceções

#12 - AGENDA



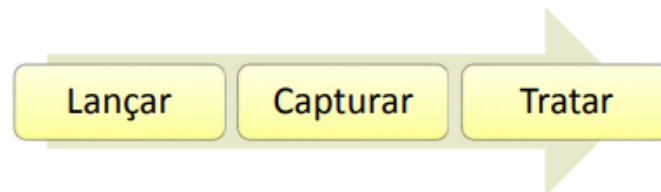
- Exceções
- Tratamento de Exceções
- Classe Exception
- Exceções mais comuns no Java
- Captura e tratamento
- Propagação de exceções
- Criando exceções

- Durante a execução, as **aplicações podem incorrer** em vários tipos de **exceções e erros** de vários graus de severidade;
- **Exceções e erros** são problemas encontrados pelo computador ao tentar **executar um comando**;
- Alguns **problemas comuns** de execução ocorrem devido:
 - **falha na aquisição de um recurso** – Exemplo: abrir um arquivo, conectar com o banco de dados;
 - **tentativa de fazer algo impossível** – Exemplo: divisão de um número por zero;
 - **acessar um índice inválido**;
 - **outras condições inválidas** – Exemplo: evocar um método de um objeto que não foi instanciado, passagem de um parâmetro com um tipo incorreto durante a execução;



A maioria dos
programadores **não** testam
todas as condições que
podem gerar erros!

- O **tratamento de exceções** fornece uma maneira de **verificar erros sem prejudicar o fluxo do programa**;
- É uma **maneira simples** e clara de lidar com as “**situações excepcionais**” que podem surgir em tempo de execução.
- Em linhas gerais, o **fluxo para tratamento** de exceções no Java ocorre em três passos:
 - **Uma exceção é lançada** – ou disparada – quando uma condição inesperada de erro é encontrada.
 - **A exceção é capturada** em algum ponto do programa.
 - **Um tratamento adequado** é realizado em algum ponto do programa.

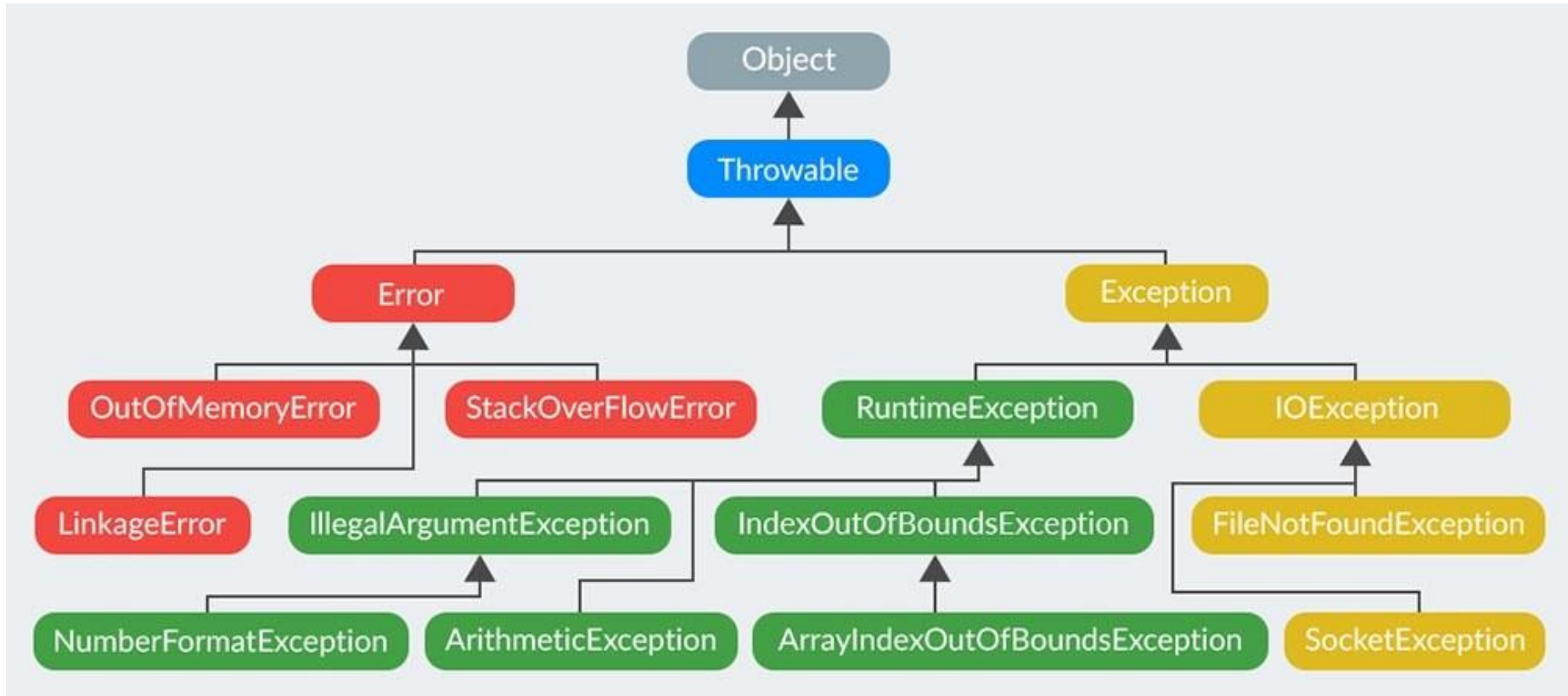




**Exceções não capturadas
provocam a finalização
do programa!**

- Uma exceção é um objeto do tipo **Exception**;
- De acordo com o polimorfismo, um objeto do tipo **Exception** pode ser uma instância de qualquer subclasse de **Exception**;
- Portanto, as exceções podem ser tratadas **polimorficamente**;
- Apenas as exceções definidas nas subclasses da classe **Exception** podem ser tratadas durante a execução do programa;
- **Erros são problemas** que não podem ser tratados durante a execução do programa, devendo o programa ser finalizado;

- O Java classifica as exceções em **duas categorias**:
 - **Checked (Exception)**
 - Possível exceção na execução de um programa;
 - Deve ser tratada pelo programador;
 - **Unchecked (Error, RuntimeException)**
 - **Error**: Erro crítico que faz com que não seja mais possível executar a aplicação;
 - **Runtime exception**: Exceção provocada por código mal escrito/testado ou pela operação inadequada do sistema – Pode ser tratada pelo programador;



Exception	Descrição
ArithmeticException	Causado por erros matemáticos, como divisão por zero
NullPointerException	Causado por referenciar um objeto vazio
NumberFormatException	Causado quando a conversão entre String e numero falha
ArrayIndexOutOfBoundsException	Causado por acessar um index inválido de um array
StringIndexOutOfBoundsException	Causado quando o programa acessa uma posição de um caractere que não existe em uma String
FileNotFoundException	Causado por tentar acessar um arquivo não existente
IOException	Causado por falha de IO, como não conseguir ler um arquivo



TRATAMENTO DE EXEÇÕES

- Para **tratar as exceções em tempo de execução**, elas devem ser **capturadas e manipuladas**;
- O Java tem duas estruturas importantes para tratamento de exceções:
 - **try-catch**
 - **try-catch-finally**
- Estas **duas estruturas** tem a finalidade de **separar o código** que **executa as tarefas desejadas**, das rotinas de **tratamento de erros** eventuais;
- Através destas diretivas é delimitado o **trecho de código** que **será monitorado pela JVM**. Quando um **erro for gerado**, o **fluxo de execução é desviado** de forma seletiva uma rotina de tratamento;
- Uma exceção pode ser **tratada** ou **simplesmente lançada**;

- O código dentro do bloco **try** representa um código que **poderá gerar uma exceção**, ou seja, um código “perigoso” para execução;
 - Um bloco **try** é seguido por um ou mais blocos **catch**;
- Caso uma **exceção seja lançada**, o bloco **try** é **finalizado** e o **fluxo de execução** procura um bloco **catch** adequado para tratar a exceção;
 - Cada bloco **catch** trata um tipo **específico de exceção**;
- Depois de executar um bloco **catch**, o **fluxo de execução do programa continua** na primeira instrução depois do último bloco **catch**;
- Uma **exceção não capturada** é uma exceção para a qual não existe nenhum bloco **catch** correspondente;

- A captura das diferentes exceções pelos blocos **catch** funciona da seguinte forma:
 - **Apenas o primeiro catch que se encaixar** com o tipo da exceção lançada é executado; por isso, as **exceções mais específicas** – de nível mais baixo na hierarquia de classes - **devem ser escritas nos primeiros catch**;
 - A exceção do tipo **Exception** é a **mais geral possível** – ancestral de todas as exceções – então um bloco catch que receba uma exceção desse tipo pode capturar qualquer tipo de exceção que for lançada (**Por isto sempre aparece por último numa sequencia de blocos catch**);
 - **Se nenhum catch capturar a exceção**, acontecerá o mesmo que aconteceria caso os comandos **try-catch** não fossem usados;


```
try {  
    //fluxo normal, código que pode gerar uma Exception  
} catch(Exception1 e) {  
    //fluxo alternativo, caso a Exception1 seja gerada  
} catch(Exception2 ex) {  
    //fluxo alternativo, caso a Exception2 seja gerada  
}
```

```
Scanner leitor = new Scanner(System.in);  
  
int dividendo = leitor.nextInt();  
  
int divisor = leitor.nextInt();  
  
try {  
    int resultado = dividendo/divisor;  
    System.out.println("Resultado: " + resultado);  
} catch (ArithmeticException e) {  
    System.out.println("Erro ao dividir");  
}
```

- No **Java 7** é possível tratar mais de uma exceção no mesmo bloco **catch**:

```
try {  
    //fluxo normal, código que pode gerar uma Exception  
} catch(Exception1 | Exception2 e) {  
    //fluxo alternativo, caso a Exception1 ou Exception2 seja gerada  
}
```

```
try {  
    int resultado = dividir(dividendo/divisor);  
    System.out.println("Resultado: " + resultado);  
} catch(ArithmeticException | IllegalArgumentException e) {  
    System.out.println("Erro ao dividir");  
}
```

- O bloco **finally** é **executado sempre**, independente de ter ocorrido ou não uma exceção;
 - Se um bloco **try** falhar - gerar uma exceção - o fluxo passará para o bloco **catch** correspondente. Após a execução do **catch**, o bloco **finally** será executado;
 - Se o bloco **try** for bem sucedido - não gerar exceção - o bloco **finally** será executado;
- É um bloco **opcional**, mas se estiver presente deve ser colocado depois do **último bloco catch**;

```
try {  
    //fluxo normal, código que pode gerar uma Exception  
}catch(Exception e) {  
    //fluxo alternativo, caso a Exception seja gerada  
}finally {  
    //fluxo normal, que sempre será executado  
}
```

```
Scanner leitor = new Scanner(System.in);  
  
try {  
    int dividendo = leitor.nextInt();  
    int divisor = leitor.nextInt();  
    int resultado = dividendo/divisor;  
    System.out.println("Resultado: " + resultado);  
} catch (ArithmeticException e) {  
    System.out.println("Erro ao dividir");  
} catch (InputMismatchException e) {  
    System.out.println("Entrada de dados inválidos");  
} finally {  
    leitor.close();  
    System.out.println("Finalizando o programa");  
}
```



PROPAGAÇÃO DE EXEÇÕES

- Um **método pode deixar de tratar uma exceção** e simplesmente **propagá-la**, ou melhor, delegar para outro método – o que invocou o segundo – o tratamento da exceção que venha a ser gerada durante sua execução;
- Para isto basta utilizar as **palavra-chave throws** seguida do **tipo de exceção** que este método pode propagar;

```
public int dividir(int x, int y) throws ArithmeticException {  
    return x/y;  
}
```



- Um **método pode propagar mais de um tipo de exceção**, basta declarar as exceções separadas por vírgula, exemplo:

```
public void gravarNorArquivo(String texto)
    throws FileNotFoundException, IOException {
    //....
}
```



- Um método pode forçar a ocorrência de uma exceção e depois lançá-la. Para isto basta utilizar as palavras-chaves **throw** e **throws**.

```
public int dividir(int x, int y) throws ArithmeticException {  
    if (y == 0){  
        throw new ArithmeticException("Não é possível dividir por 0");  
    }  
    return x/y;  
}
```



CRIANDO EXCEÇÕES

- É possível criar suas próprias exceções, para isto basta criar uma classe que descenda da classe **Exception**;

```
public class DivisaoPorZeroException extends Exception {  
  
    public DivisaoPorZeroException() {  
        super("Não é possível dividir por 0");  
    }  
}
```

Implemente o **diagrama de classes** exibido a seguir. Depois modifique as assinaturas dos métodos **sacar**, **depositar** e **setValorLimite** de forma que possam lançar uma exceção da classe **Exception**.

ContaCorrente
- saldo : double - limite : double
+ ContaCorrente() + ContaCorrente(saldo : double, limite : double) + sacar(valor : double) : void + depositar(valor : double) : void

- Uma exceção deverá ser lançada toda vez que os métodos **sacar**, **depositar** e **setValorLimite** receberem um valor negativo.

- **Uma exceção deverá ser lançada** toda vez que o método **sacar** receber um valor maior que o permitido pelo limite.
- **Ao criar e lançar uma exceção** não esqueça de definir uma mensagem de erro associado a ela, para isto basta passar uma String contendo a mensagem da exceção no construtor.
- Crie um programa chamado **TesteContaCorrente** para testar a classe **ContaCorrente**. Não esqueça de implementar o tratamento de exceções na classe **TesteContaCorrente**. Nos blocos de tratamento das exceções exibir para o usuário a exceção ocorrida – utilize o método **getMessage()** da **Exception** declarada no **catch**.
- Instancie um objeto da classe **ContaCorrente**, utilize os métodos **sacar**, **depositar** e **setLimite()** e cause exceções.

Copyright © 2020 - 2025
Prof. Thiago T. I. Yamamoto

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).